UNITÀ 3 · APPLICAZIONI JAVA PER LA COMUNICAZIONE IN RETE

Il ping multiplo: tutta la rete in un secondo

Controllare un host è facile. Controllarne duecento senza aspettare quattro minuti richiede thread, pool e un po' di attenzione alla concorrenza.

PER COMINCIARE

Il laboratorio ha 30 PC. Come scopri quali sono accesi?

SONDAGGIO

- **A** Faccio il giro e guardo gli schermi
- **B** Un ping alla volta, a mano
- **C** Un programma che li controlla tutti insieme
- **D** Chiedo al tecnico, che ha un sistema di monitoraggio

IL PROBLEMA

Il tempo perso è tutto nell'attesa

Una rete /24, tutti gli host spenti, timeout 1 s, in sequenza

$$254 \times 1\ \text{s} = 254\ \text{s}$$

Stessa rete, 32 thread in parallelo

$$\lceil 254 / 32 \rceil \times 1\ \text{s} = 8\ \text{s}$$

- Un controllo passa quasi tutto il tempo ad **aspettare la rete**, non a calcolare.
- Mentre un thread aspetta, la CPU è libera: altri thread possono lanciare altri controlli.
- Compiti così si dicono I/O-bound: il parallelismo li accelera anche su un PC con pochi core.

FAI IL CONTO

100 host spenti, timeout 2 s, un pool di 25 thread. Quanto dura la scansione?

QUIZ

- **A** Circa 2 secondi
- **B** Circa 50 secondi
- **C** Circa 200 secondi
- **D** Circa 8 secondi

GLI ATTREZZI DI JAVA

Dai thread a mano al pool

Thread

`new Thread(r).start()`

Un thread per compito, creato a mano. Il risultato va passato con variabili condivise.

ExecutorService

`pool.submit(compito)`

Un gruppo di N thread riusati. Riceve compiti e li mette in coda.

Future

`futuro.get()`

Una "ricevuta": **get()** aspetta che il compito finisca e ne restituisce il risultato.

Un **Callable** è come un Runnable che **restituisce un valore** (e può lanciare eccezioni): perfetto per "controlla questo host e dimmi com'è andata".

IL CODICE · 1

Il compito: controllare un host e restituire un esito

```
record Esito(String host, boolean vivo, long ms) {}

static Esito controlla(String host) {
    long t0 = System.nanoTime();
    try {
        boolean ok = InetAddress.getByName(host).isReachable(1000);
        return new Esito(host, ok, (System.nanoTime() - t0) / 1_000_000);
    } catch (IOException e) {          // host sconosciuto, rete assente...
        return new Esito(host, false, -1);
    }
}
```

Un `record` (Java 16+) è una classe immutabile con costruttore, getter `host()`, `vivo()`, `ms()` e `toString()` già pronti.

IL CODICE · 2

Il pool: un compito per host, i risultati in ordine

```
List<String> hosts = Files.readAllLines(Path.of("hosts.txt"));
long t0 = System.nanoTime();
try (ExecutorService pool = Executors.newFixedThreadPool(32)) {
    List<Future<Esito>> futuri = new ArrayList<>();
    for (String h : hosts)
        futuri.add(pool.submit(() -> controlla(h)));    // parte subito
    for (Future<Esito> f : futuri)
        System.out.println(f.get());                     // aspetta quel risultato
}                                    // close(): attende la fine di tutti
System.out.println("Tempo: " + (System.nanoTime() - t0) / 1_000_000 + " ms");
```

I risultati escono nell'**ordine della lista**, anche se i compiti finiscono in ordine diverso: `get()` aspetta proprio quel futuro.

AL COMPUTER · 25 minuti

Sequenziale contro parallelo

1. Nella cartella **u3-multiplo** crea **hosts.txt**: 127.0.0.1, il gateway e da `192.0.2.1` a `192.0.2.10`, uno per riga.
2. Scrivi **PingMultiplo.java** con il codice delle due slide.
3. Scrivi **Sequenziale.java**: stesso file, un `for` che chiama `controlla` uno dopo l'altro.
4. Esegui entrambi e annota il tempo totale in **tempi.txt**.
5. Prova il pool con 1, 4 e 12 thread.

Controllo: i 10 indirizzi 192.0.2.x non rispondono. Sequenziale: circa 10 s. Pool da 12: poco più di 1 s. Pool da 4: circa 3 s (10 compiti lenti in 3 turni).

QUIZ A TRABOCCHETTO

4 thread fanno `n++` 100 000 volte ciascuno su un `static int n`. Che cosa stampa?

QUIZ

A Sempre 400 000

B Un numero fino a 400 000, spesso diverso a ogni esecuzione

C Sempre 100 000

D Non compila: serve synchronized

PUZZLE

Abbina ogni attrezzo al problema che risolve

AtomicInteger	---	Un contatore condiviso che non perde incrementi
ConcurrentHashMap	---	Un solo thread alla volta dentro un blocco di codice
Future.get()	---	Ripetere un compito ogni N secondi
synchronized	---	Una mappa che più thread possono aggiornare insieme
ScheduledExecutorService	---	Aspettare il risultato di un compito già lanciato

JAVA 21

Virtual thread: migliaia di attese, pochi thread veri

```
try (var pool = Executors
        .newVirtualThreadPerTaskExecutor()) {
    // un virtual thread per ogni compito
}
```

64 host che non rispondono, timeout 1 s, 4 core	Tempo
Pool fisso da 64 + isReachable	~1 s
Virtual thread + isReachable	~16 s
Virtual thread + `Socket.connect`	~1 s

Un virtual thread costa pochissimo: la JVM lo "parcheggia" mentre aspetta un socket e usa il thread vero per altro.

> **Ma** `isReachable` è una chiamata nativa che blocca il thread vero: con 4 core se ne fanno solo 4 alla volta.

CONTROLLARE UN SERVIZIO

L'host risponde. Ma il sito funziona?

```
static boolean aperta(String host, int porta) {
    try (Socket s = new Socket()) {
        s.connect(new InetSocketAddress(host, porta), 1000);   // timeout 1 s
        return true;
    } catch (IOException e) {
        return false;
    }
}
```

Connessione riuscita

Il servizio ascolta su quella porta.

Connection refused

Risposta immediata: host acceso, porta chiusa.

Timeout

Nessuna risposta: host spento o porta filtrata.

AL COMPUTER · 20 minuti

Porte aperte, chiuse, filtrate

1. Crea **PortaAperta.java** con il metodo `aperta` e un `main` che usa `newVirtualThreadPerTaskExecutor()`.
2. In un secondo terminale avvia un web server di prova: `jwebserver -p 8000` (è nel JDK).
3. Controlla `127.0.0.1` sulle porte 8000 e 8001, poi `192.0.2.1` sulla 80.
4. Per ogni caso stampa il tempo e il nome dell'eccezione: `e.getClass().getSimpleName()`.
5. Annota i risultati in **porte.txt**.

Controllo: 8000 aperta in pochi ms; 8001 `ConnectException` (refused) quasi subito; 192.0.2.1:80 `SocketTimeoutException` dopo circa 1000 ms.

MENTI CHE CI HANNO PRECEDUTO

Perché controllare continuamente

> **Un sistema distribuito è un sistema in cui il guasto di un computer di cui non conoscevi nemmeno l'esistenza può rendere inutilizzabile il tuo.**
>
> **Leslie Lamport**, informatico, premio Turing 2013, in una email del 1987

Il registro elettronico, la stampante, il DNS, il Wi-Fi: un monitor ci dice **prima degli utenti** che qualcosa si è rotto.

IL MONITOR

Ogni 5 secondi, tutti i servizi, un allarme se cambia

```
static final Map<String, Boolean> stato = new ConcurrentHashMap<>();

static void controlla(String voce, PrintWriter csv) {        // voce = "host:porta"
    String[] p = voce.split(":");
    boolean su = PortaAperta.aperta(p[0], Integer.parseInt(p[1]));
    Boolean prima = stato.put(voce, su);                      // stato precedente
    synchronized (csv) { csv.println(ora() + ";" + voce + ";" + (su ? "SU" : "GIU")); csv.flush(); }
    if (prima != null && prima != su)
        System.out.println("ALLARME " + voce + (su ? " torna SU" : " va GIU"));
}
// nel main
timer.scheduleAtFixedRate(() -> {
    for (String v : voci) pool.submit(() -> controlla(v, csv));
}, 0, 5, TimeUnit.SECONDS);
```

AL COMPUTER · 30 minuti

Il monitor del laboratorio

1. Crea **servizi.txt** con `127.0.0.1:8000`, `127.0.0.1:5000` e i servizi del laboratorio indicati dal docente.
2. Scrivi **Monitor.java** partendo dalla slide precedente.
3. Avvialo, poi in un altro terminale avvia e ferma `jwebserver -p 8000` un paio di volte.
4. Apri **monitor.csv** con LibreOffice Calc (separatore `;`).

Controllo: quando avvii jwebserver compare `ALLARME 127.0.0.1:8000 torna SU` entro 5 secondi; quando lo fermi, `va GIU`. Nel CSV ogni riga ha ora, servizio e stato.

PUZZLE

Metti in ordine i passi di una scansione parallela

1. Con get() raccogli gli esiti nell'ordine della lista
2. Per ogni host, submit di un compito e salva il Future
3. Crea il pool di thread
4. Chiudi il pool e stampa il tempo totale
5. Leggi la lista degli host da hosts.txt
6. Ogni compito misura e restituisce un Esito

REGOLE DEL GIOCO

Scansionare una rete non è un gioco innocente

Mai

- Scansionare reti di altri: aziende, vicini di casa, server su Internet.
- Scansionare la rete della scuola senza il permesso del responsabile.

Sì

- Il tuo PC, la tua rete di casa, la rete del laboratorio con il docente.
- Gli indirizzi di documentazione 192.0.2.x, che non portano a nessuno.

I sistemi di difesa (IDS, firewall) riconoscono una scansione come la **prima fase di un attacco**, la ricognizione: il tuo programma può far scattare un allarme vero.

IL MONITORAGGIO VERO

Che cosa usano le aziende





ALL'ESAME

Progettare un sistema di monitoraggio

1. **Che cosa** controlli: host (ICMP) o servizi (TCP, HTTP)?
2. **Ogni quanto** e con quale timeout: il caso peggiore dura [host / thread] × timeout.
3. **Come**: pool fisso o virtual thread, e perché.
4. **Dove** salvi lo storico e **chi** riceve gli allarmi.

Frase da orale

"Il controllo è I/O-bound: i thread passano il tempo ad aspettare, quindi il parallelismo riduce il tempo totale anche con pochi core. Ho misurato prima di scegliere."

BIGLIETTO D'USCITA

Completa prima di uscire

Un compito che passa il tempo ad aspettare la rete si dice _____.

Il metodo `submit` di un ExecutorService restituisce un _____.

Due thread che fanno `n++` sulla stessa variabile creano una race _____.

120 host spenti, timeout 1 s, 40 thread: la scansione dura circa ____ secondi.

Un "TCP ping" verso una porta chiusa riceve subito un errore connection _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- In sequenza: host × timeout. In parallelo: [host / thread] × timeout
- ExecutorService, Callable e Future: lanciare tutto, poi raccogliere
- Variabili condivise = race condition: Atomic, Concurrent, oppure niente condivisione
- Virtual thread: ottimi con i socket, inutili con isReachable
- TCP ping: refused = porta chiusa, timeout = spento o filtrato
- Si scansiona solo dove si è autorizzati

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.